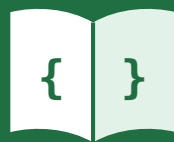


---

# Module 13 — Reactive Programming and WebFlux

Exercises

Spring Boot Edu



2026-09-24

## Contents

|                                                              |          |
|--------------------------------------------------------------|----------|
| <b>How to Work</b>                                           | <b>2</b> |
| <b>Exercise 1 — A Reactive Book API (Easy)</b>               | <b>2</b> |
| <b>Exercise 2 — A Live Stock Feed with SSE (Medium)</b>      | <b>3</b> |
| <b>Exercise 3 — A Product Page from Three Sources (Hard)</b> | <b>3</b> |
| <b>Extra Challenge (Optional)</b>                            | <b>4</b> |

## How to Work

1. The starter code is in `modules/13-reactive/exercise/`. Find the `TODO` comments.
2. The exercises need no Docker: the given services keep their data in memory, but are fully reactive.
3. Run the tests:

```
./mvnw -Pexercises -pl modules/13-reactive/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/13-reactive/solution/`.

## Exercise 1 — A Reactive Book API (Easy)

**Goal:** offer the books of the given BookStore as a REST API with **functional endpoints**.

| Request                  | Answer                                |
|--------------------------|---------------------------------------|
| GET /api/books           | 200, all books                        |
| GET /api/books/{isbn}    | 200 with the book, or 404             |
| POST /api/books          | 201 with Location: /api/books/{isbn}  |
| DELETE /api/books/{isbn} | 204, or 404 if the book did not exist |

**Tasks** (`exercisel.BookRoutes`):

- `TODO 1a` — a `RouterFunction<ServerResponse>` bean with the prefix `/api/books`.
- `TODO 1b` — the two GET routes.
- `TODO 1c` — POST.
- `TODO 1d` — DELETE.

**Hints:**

- Lesson section 3.4: `route().path("/api/books", api -> api.GET(...).POST(...)).build()`.
- `books.find(isbn)` is empty for an unknown ISBN: `switchIfEmpty(ServerResponse.notFound()).build()`.
- `books.delete(isbn)` emits true or false: decide with `flatMap(deleted -> deleted ? ... : ...)`.
- Name the bean method differently from the class (e.g. `bookRouter`), otherwise the two bean names clash.

**Acceptance criteria:** all 4 tests in `ExerciselTest` pass.

**Estimated time:** 25 minutes

## Exercise 2 — A Live Stock Feed with SSE (Medium)

**Goal:** a warehouse screen shows stock levels live. Every change is pushed to all open screens. A screen may follow a single book.

StockFeed is given: `publish(level)` sends a change, `changes()` is a hot Flux of all changes.

**Tasks** (`exercise2.StockController`):

- TODO 2a — PUT `/api/stock/{isbn}` with the new quantity as JSON body: publish a `StockLevel`, answer 204.
- TODO 2b — GET `/api/stock/stream` as text/event-stream: every change as a Server-Sent Event.
- TODO 2c — with `?isbn=...`, only the changes of that book.
- TODO 2d — send a comment event first, so that the client gets the response headers at once.

**Hints:**

- Lesson section 3.6 shows the complete pattern (`ServerSentEvent.builder(...), startWith(...)`).
- `@RequestParam(required = false) @Nullable String isbn` and `filter(...)`.
- `@RequestBody int quantity` reads a plain JSON number such as 7.

**Acceptance criteria:** all 3 tests in `Exercise2Test` pass.

**Estimated time:** 30 minutes

## Exercise 3 — A Product Page from Three Sources (Hard)

**Goal:** the product page shows price, stock and rating. Each comes from a different service, and each takes about 300 ms. Load them **in parallel**, so the page takes 300 ms and not 900 ms. The rating is optional: the page must still work when it is missing, failing or too slow.

**Tasks** (`exercise3.ProductPage.load`):

- TODO 3a — the rating: no rating, an error, or more than 1 second → 0.0.
- TODO 3b — ask the three services at the same time and combine the answers into a `ProductView`.
- TODO 3c — if there is no price (unknown book), the result is empty.

**Hints:**

- Lesson section 3.5: `Mono.zip(a, b, c).map(t -> ...t.getT1()...)`. `zip` is empty as soon as one source is empty, which gives you TODO 3c for free.

- `timeout(Duration.ofSeconds(1)), onErrorReturn(0.0), defaultIfEmpty(0.0)`. Think about the order: a timeout is an error, too.
- The tests use virtual time: `expectNoEvent(299 ms)` fails if your code loads the sources one after another.

**Acceptance criteria:** all 5 tests in `Exercise3Test` pass.

**Estimated time:** 40 minutes

## Extra Challenge (Optional)

Add `GET /api/products/{isbn}` to the application that serves the `ProductPage` result (404 for an empty result). Write a price service with `WebClient` that calls the lesson's `/api/books/{isbn}` endpoint. Then measure with `curl -w "%{time_total}"` how long the page takes when one service is down.